MNIST Handwritten Digits Classification

Using Bidirectional Long Short-Term Memory (BiLSTM)

Project Type	Deep Learning — Image Classification
Dataset	MNIST Handwritten Digits (0–9)
Language / Libraries	Python, TensorFlow/Keras, NumPy, Pandas, Matplotlib, scikit-learn
Environment	Google Colab
Model	Bidirectional LSTM neural network

Project Overview

This project recognizes handwritten digits from 0 to 9 using a Bidirectional LSTM. Each grayscale image is 28 × 28 pixels and is represented as a sequence of 28 rows, each containing 28 normalized pixel values. The notebook loads the MNIST dataset through TensorFlow's built-in dataset loader, so no Kaggle API token or kaggle.json file is required for this version.

Documentation prepared for a machine-learning / deep-learning portfolio.

1. Introduction

Handwritten digit recognition is a standard computer-vision task in which a model predicts the digit shown in an image. This project applies a recurrent neural network—specifically a Bidirectional Long Short-Term Memory network—to image rows treated as sequential input.

2. Project Objectives

- Load MNIST directly in Google Colab without manually downloading a ZIP file.
- Normalize pixel values to the range 0–1.
- Train a BiLSTM model to classify ten digit categories.
- Evaluate test performance with accuracy, loss, and a classification report.
- Plot training/validation accuracy and loss curves.
- Create a confusion matrix and inspect actual-versus-predicted images.
- Export predictions to CSV and save the trained model.

3. Dataset Description

MNIST contains 70,000 grayscale images of handwritten digits. The standard TensorFlow split contains 60,000 training images and 10,000 test images. Each image is 28 × 28 pixels and its label is an integer from 0 to 9. In the notebook, 10% of the training portion is used for validation.

Property	Description
Total images	70,000
Training portion	60,000 (90% training / 10% validation during fit)
Test portion	10,000
Image dimensions	28 × 28 pixels
Color	Grayscale
Classes	10 digits (0–9)
Model input	28 sequential steps × 28 features

4. Tools and Technologies

- Python: implementation language.
- TensorFlow/Keras: dataset loading, model definition, training, and evaluation.
- NumPy and Pandas: numerical operations and CSV export.
- Matplotlib: sample images and training/evaluation plots.
- scikit-learn: confusion matrix and classification metrics.
- Google Colab: cloud notebook execution.

5. Model Architecture

The model receives an input with shape (28, 28). Each image is treated as a sequence of 28 rows, with 28 pixel values per row.

Layer	Role
Input (28, 28)	Receives the sequence of image rows.
Bidirectional LSTM (64, return_sequences=True)	Learns forward and backward patterns across rows.
Layer Normalization	Normalizes intermediate activations.
Dropout (0.25)	Helps reduce overfitting.
Bidirectional LSTM (64)	Produces a compact sequence representation.
Dropout (0.35)	Adds regularization.
Dense (64, ReLU)	Learns nonlinear feature combinations.
Dropout (0.20)	Additional regularization.
Dense (10, Softmax)	Outputs probabilities for digits 0–9.

6. Training Configuration

Setting	Value
Optimizer	Adam
Loss	Sparse categorical cross-entropy
Metric	Accuracy
Epochs	10
Batch size	128
Validation	10% of training data
Learning-rate scheduler	ReduceLROnPlateau monitors validation loss

7. Evaluation and Results

- Test accuracy and loss are calculated on the 10,000 held-out test images.
- Accuracy and loss curves show training and validation trends.
- The confusion matrix shows which digits are correctly classified or confused.
- The classification report provides precision, recall, and F1-score for each digit.
- Sample predictions visually compare actual labels with predicted labels.

No numeric accuracy is stated here because the notebook's actual run output was not supplied. Insert the measured test accuracy and loss after training.

8. Generated Files

File name	Purpose
mnist_lstm_predictions.csv	Actual digit, predicted digit, correctness flag, and confidence.

File name	Purpose
mnist_bidirectional_lstm.keras	Saved trained Keras model.

9. How to Run

1. Open Google Colab and create a notebook.
2. Run the project cells in order, starting with imports and dataset loading.
3. Wait for model training to finish.
4. Review the accuracy/loss graphs, confusion matrix, and classification report.
5. Run the CSV export and model-saving cells.
6. Use the provided download cells to download the prediction CSV and saved model.

10. Limitations

- LSTM networks are designed primarily for sequential data; CNNs are generally a more natural choice for images.
- Treating image rows as a sequence may not capture two-dimensional spatial patterns as effectively as convolutional layers.
- Actual results depend on runtime, model initialization, and training; accuracy should be reported from the executed notebook.
- This implementation loads MNIST using TensorFlow, not Kaggle, so Kaggle credentials are not required.

11. Conclusion

The project demonstrates a complete deep-learning workflow for handwritten digit recognition with a Bidirectional LSTM. It includes direct dataset loading, preprocessing, training, evaluation, visualization, prediction export, and model saving. A useful next step is to compare the BiLSTM with a CNN baseline on the same test set.

12. Future Improvements

- Compare BiLSTM, CNN, and CNN–LSTM models.
- Tune LSTM units, dropout, batch size, and learning rate.
- Add custom-image prediction for a user's handwritten digit.
- Record measured test accuracy, test loss, and per-class metrics.